Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

Synthetic Control and Experimental Design in One Validated Interface: The `mlsynth` Package for Python

Jared Greathouse
Georgia State University

Abstract

Synthetic control and its many descendants have become a default tool for estimating the effect of a policy, product, or intervention that lands on a small number of aggregate units. The methodological literature has since fractured into dozens of variants – robust, penalized, factor-model, proximal, forward-selected, and matrix-completion estimators, alongside a newer family of experimental-design methods that choose which units to treat before an intervention is run. Each variant typically arrives as a separate package, with its own data conventions, its own notion of a result, and its own (or no) inference. This paper introduces `mlsynth`, a strongly typed Python library that places more than forty of these estimators behind a single data-preparation step, a single validated configuration-and-fit interface, and a single standardized result object. We argue that these methods are not forty unrelated tools but one family built on a shared factor-model projection, so a unified interface is what makes them directly comparable – and what lets the library reach a capability the surrounding ecosystem never packaged for practitioners: experimental design. We demonstrate the library with three worked illustrations, organised by the regime they address. The first re-estimates the canonical California tobacco-control study with three synthetic-control variants and attaches Cattaneo, Feng, and Titiunik prediction intervals. The second relaxes the no-interference assumption, running four spillover-aware estimators – each otherwise a separate codebase – across two case studies through a single dispatcher. The third reproduces, to within numerical tolerance, Meta's published `GeoLift` geo-experiment walkthrough, and shows how the same interface exposes the broader experimental-design family.

Keywords: synthetic control, causal inference, panel data, experimental design, Python.

1. Introduction

A recurring problem in applied work is estimating what would have happened to a single unit – a state, a country, a market, a product line – had some intervention not occurred. Randomized

experiments are usually impossible at this level of aggregation: there is one California, and it either passed an anti-smoking proposition or it did not. The synthetic control method (Abadie and Gardeazabal 2003; Abadie, Diamond, and Hainmueller 2010) answered this by constructing the missing counterfactual as a weighted average of untreated units – a *synthetic* California assembled from other states – chosen so that the weighted donors track the treated unit over a pre-intervention training window. The method has become one of the most widely used tools in the program-evaluation toolkit (Abadie 2021), and the original software, the `Synth` package (Abadie, Diamond, and Hainmueller 2011), was itself published in this journal.

In the fifteen years since, the single method has become a research program. The simplex-weighted original now sits alongside robust low-rank variants (Amjad, Shah, and Shen 2018), penalized and regularized weights, augmented estimators that debias the fit with an outcome model (Ben-Michael, Feller, and Rothstein 2021), factor-model and interactive-fixed-effects approaches (Bai 2009; Xu 2017), matrix completion (Athey, Bayati, Doudchenko, Imbens, and Khosravi 2021), the panel-data forward-selection approach of Hsiao, Ching, and Wan (2012), proximal and surrogate methods, and the synthetic-difference-in-differences estimator (Arkhangelsky, Athey, Hirshberg, Imbens, and Wager 2021). A parallel and newer literature inverts the question entirely: instead of building a counterfactual after an intervention, *experimental-design* methods choose which units to treat, and for how long, so that a credible synthetic control will exist when the experiment is over (Doudchenko and Imbens 2016; Doudchenko, Khosravi, Pouget-Abadie, Lahaie, Lubin, Mirrokni, Spiess, and Imbens 2021). Meta's `GeoLift` framework (Meta Open Source 2024) brought this design-first idea to industry practice for geographic marketing experiments.

This proliferation has a cost for the practitioner. Each variant typically arrives as a separate package – often in a different language, with its own way of reshaping a panel, its own argument names, its own definition of what a "result" is, and its own inference procedure, when it has one at all. Comparing two estimators on the same data can mean reformatting the data twice, learning two APIs, and hand-reconciling two incompatible notions of a treatment effect. Worse, the gap between the methods literature and usable software is uneven: some influential estimators have no maintained implementation, and the experimental-design family in particular has had almost no packaged presence for applied users.

Naming the leading tools makes the cost concrete. The original method ships as the R package `Synth` (Abadie *et al.* 2011) and, in Stata, as `synth2` (Yan and Chen 2023), which adds placebo and leave-one-out diagnostics; the panel-data regression / regression-control approach has the Stata command `rcm` (Yan and Chen 2022); micro-level synthetic control has the R package `microsynth` (Robbins and Davenport 2021); synthetic difference-in-differences has the R `synthdid` and the Stata `sdid` (Arkhangelsky *et al.* 2021; Clarke, Pailañir, Athey, and Imbens 2023); prediction-interval inference has `scpi` (Cattaneo, Feng, Palomba, and Titiunik 2025); forward-selected estimation has `fsPDA` (Shi and Huang 2023); and design-first geo-experiments have Meta's `GeoLift` (Meta Open Source 2024). Each is a capable tool and `mlsynth` is a critique of none of them. But they are spread across two languages – several only in the commercial Stata – with no shared data format, no shared notion of a result, and no way to run one against another on the same panel without redoing the work each time. An applied researcher who wanted to compare a simplex control, a panel-data regression, and a synthetic difference-in-differences estimate on one dataset had, until recently, to install several packages, reshape the data several ways, learn several syntaxes, and reconcile several output formats by

hand.

This paper introduces `mlsynth`, a Python (Harris, Millman, van der Walt *et al.* 2020) library that addresses this fragmentation directly. The package ships 47 estimators behind one data-preparation routine, one validated configuration-and-fit interface, and one standardized result object. Our central argument is that this consolidation is worth having – that these are not 47 unrelated tools but one family. As we make precise in Section 2, the vast majority of these estimators reduce to a common operation: projecting a treated unit's outcome onto a low-dimensional factor structure recovered from the donor pool. Placing them behind one interface is therefore not mere packaging convenience. It is what makes the estimators directly comparable on identical inputs, what lets each one inherit the same validated data handling and the same modern inference (conformal intervals and prediction intervals), and what lets a single library span both the estimation question and the design question that the wider ecosystem has never packaged together. Concretely, a researcher can now fit a `Synth`-style simplex control, an `rcm`-style panel-data regression, and a `synthdid`-style difference-in-differences design on one data frame, compare them directly, attach prediction-interval or conformal inference to each, and – in the same library and the same syntax – design the experiment that a synthetic control will later analyze. That combined workflow previously spanned several packages across two languages, where it was available at all.

The aim of `mlsynth` is to make this catalog of estimators accessible – free, and as simple to call as the toolkit can be made. This paper is written to match that aim, and it is worth stating its scope plainly. It does not re-derive the forty-odd methods it ships; each carries its own optimization problem, inference theory, and proofs, and those belong to the source papers and to the per-estimator documentation (for example https://mlsynth.readthedocs.io/en/latest/sdid.html), which we cite rather than reproduce. Nor is there anything special about the three estimators we use to illustrate the library below: none is privileged over the dozens we do not show, and a different three would have served as well. That very arbitrariness is the argument. The only reason these methods can be demonstrated side by side, swapped for one another with a one-line edit, is that they share a common input contract and a common interface – which is precisely what the library exists to provide.

The remainder of the paper is organized as follows. Section 2 develops the shared factor-model view and the resulting taxonomy of estimators. Section 3 describes the library's architecture: the canonical data-preparation contract, the Pydantic-validated configuration objects (Colvin and Pydantic Contributors 2024), and the standardized result hierarchy. Section 4 works through an estimation illustration on the California tobacco-control data, comparing three synthetic-control designs and attaching prediction intervals. Section 6 works through a design illustration that reproduces Meta's `GeoLift` walkthrough and situates it within the broader design family. Section 7 records computational and reproducibility details, and Section 8 concludes.

2. A unified view of synthetic control

2.1. The shared model

Consider a balanced panel of N units observed over T periods. Let y_{it} denote the outcome of unit i in period t. A single treated unit, indexed $i = 1$, receives an intervention at time $T_0 + 1$; the remaining $N-1$ units, the *donor pool*, are never treated. Collect the donor outcomes in the $T \times (N-1)$ matrix $\mathbf{Y}_0$ and the treated unit's outcomes in the T-vector $\mathbf{y}_1$. The first T_0 rows are the pre-intervention window; the remaining $T_1 = T - T_0$ rows are the post-intervention window.

Almost every estimator in `mlsynth` rests on the assumption that outcomes are governed by a low-dimensional factor model,

$$y_{it} \;=\; \delta_t \;+\; \boldsymbol{\lambda}_i^{\top}\mathbf{f}_t \;+\; \varepsilon_{it}, \tag{1}$$

where $\mathbf{f}_t$ is an r-vector of unobserved common factors (with r much smaller than N), $\boldsymbol{\lambda}_i$ is unit i's vector of factor loadings, δ_t is a common time effect, and ε_{it} is idiosyncratic noise. The substantive content of Equation 1 is that units differ only through a handful of latent characteristics $\boldsymbol{\lambda}_i$. If the treated unit's loadings $\boldsymbol{\lambda}_1$ can be written as a combination of the donors' loadings, then a set of unit weights $\mathbf{w}$ that reproduces the treated unit's pre-treatment outcomes will also reproduce its factor loadings, and hence its untreated potential outcome after treatment. The synthetic control

$$\hat{y}_{1t} \;=\; \mathbf{Y}_{0,t}\,\mathbf{w}, \qquad t > T_0, \tag{2}$$

is then an estimate of the missing counterfactual, and the treatment effect in period t is the gap $\tau_t = y_{1t} - \hat{y}_{1t}$.

The weights themselves are, for almost every estimator in the library, the solution of one common optimization program. Writing $\mathbf{Y}_0$ and $\mathbf{y}_1$ now for their pre-treatment (T_0-row) blocks, the donor weights solve

$$\hat{\mathbf{w}} \;\in\; \underset{\mathbf{w}\in\mathcal{C}}{\arg\min}\;\; \mathcal{L}(\mathbf{Y}_0, \mathbf{y}_1, \mathbf{w}) \;+\; \mathcal{P}(\mathbf{w}) \qquad \text{subject to} \qquad \mathcal{B}(\mathbf{Y}_0, \mathbf{y}_1, \mathbf{w}) \;\le\; \tau. \tag{3}$$

Here $\mathcal{L}$ is a loss measuring pre-treatment fit, $\mathcal{P}$ is a penalty that shapes the weights, $\mathcal{C}$ is the admissible set the weights must lie in, and $\mathcal{B} \le \tau$ is an optional set of balance conditions with relaxation level τ; either the loss or the balance operator may be absent, but not both. Classical synthetic control (Abadie *et al.* 2010) is the specialization with a squared-error loss $\mathcal{L} = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2$, no penalty ($\mathcal{P} = 0$), no separate balance constraint, and the admissible set fixed to the unit simplex $\mathcal{C}_\Delta = \{\mathbf{w} \ge \mathbf{0} : \mathbf{1}^{\top}\mathbf{w} = 1\}$, which yields an interpretable, sparse convex combination of donors. The rest of the family is, to a first approximation, a catalog of other choices for the four objects in Equation 3 – most visibly the admissible set (Table 1):

Admissible set $\mathcal{C}$	Definition	Reading
Simplex	$\mathbf{w} \ge \mathbf{0},\ \mathbf{1}^{\top}\mathbf{w} = 1$	classical convex SC
Non-negative	$\mathbf{w} \ge \mathbf{0}$	weights need not sum to one
Affine	$\mathbf{1}^{\top}\mathbf{w} = 1$	signed weights, extrapolation allowed
Unit box	$\mathbf{w} \in [0,1]^{N-1}$	each weight bounded on its own

Admissible set $\mathcal{C}$	Definition	Reading
Unconstrained	$\mathbf{w} \in \mathbb{R}^{N-1}$	ordinary regression onto donors

Table 1: Admissible weight sets $\mathcal{C}$ that different synthetic-control estimators impose in Equation 3.

Penalties $\mathcal{P}$ supply the rest: an ℓ_2 term gives ridge-augmented weights, an ℓ_1 term gives a sparse selection of donors, and an intercept b_0 – folded in by augmenting $\mathbf{Y}_0$ with a column of ones and left unpenalized – absorbs level differences between the treated unit and its donors. Moving the fit from the objective into the constraint $\mathcal{B} \leq \tau$ recovers the balancing and relaxation estimators, where one minimizes a norm of $\mathbf{w}$ subject to approximate moment matching.

What then separates the estimators is *how* the weights $\mathbf{w}$ (or, more generally, the projection onto the donor span) are obtained, and what is assumed about Equation 1. Beyond the choice of admissible set and penalty in Equation 3, members of the family differ in how they treat the donor matrix itself:

- Penalized and ridge-augmented weights trade a small amount of in-sample fit for stability when donors are collinear (Ben-Michael *et al.* 2021).
- Principal-component and robust variants first denoise $\mathbf{Y}_0$ by truncating its singular-value spectrum – estimating $\mathbf{f}_t$ directly – before fitting the weights (Amjad *et al.* 2018), which is exactly the low-rank reading of Equation 1.
- Forward-selection and best-subset approaches, following the panel-data method of Hsiao *et al.* (2012), choose a small set of donors by an information criterion rather than weighting all of them.
- Factor and interactive-fixed-effects estimators fit Equation 1 explicitly (Bai 2009; Xu 2017), and matrix-completion methods recover the full low-rank outcome matrix (Athey *et al.* 2021).

Seen this way, the catalog is a set of choices along a few axes – the constraint on the weights, whether the donor matrix is denoised first, whether donors are selected or weighted, and how the common time structure is handled – rather than a list of unrelated algorithms. This is precisely why one interface is the right abstraction: the inputs ($\mathbf{Y}_0$, $\mathbf{y}_1$, T_0) and the output (a counterfactual path and its gap) are common to all of them.

2.2. From estimation to design

The factor view also makes sense of the experimental-design family. If a credible synthetic control requires donors whose loadings can reconstruct the treated unit, then – before running an experiment – one can ask which candidate units *should* be treated so that the remaining units form a good donor pool, and how long the experiment must run to detect an effect of a given size. The design-first methods (Doudchenko and Imbens 2016; Doudchenko *et al.* 2021) solve exactly this problem, turning the estimation machinery around: the same fit quality that validates a synthetic control after the fact becomes the objective that selects treatment units

beforehand. mlsynth treats design as a first-class peer of estimation rather than a separate concern, and Section 6 demonstrates it.

3. Architecture

The library is organized around three contracts that every estimator shares: a single data-preparation step, a validated configuration object, and a standardized result. The design goal is that learning one estimator teaches the user every estimator.

3.1. One data contract

Panel data arrives in many shapes. Rather than ask each estimator to reshape it, mlsynth routes all ingestion through one routine, dataprep(), which takes a long (tidy) data frame together with the names of its outcome, unit, time, and treatment columns, and returns a canonical bundle: the wide outcome matrix, the treated vector $\mathbf{y}_1$, the donor matrix $\mathbf{Y}_0$, and the pre- and post-treatment period counts T_0 and T_1. Every estimator consumes this same bundle, so the user reshapes the data conceptually once – by naming four columns – and the heavy lifting of pivoting, alignment, and validation is shared and tested in one place.

The implication worth dwelling on is what the user does *not* have to specify. A single binary treatment indicator carries all the structure dataprep() needs: the unit whose indicator ever turns on is the treated unit, the period in which it first turns on is T_0+1, and everything before that is the pre-treatment window. The user never names the treated unit, never states the intervention date, and never declares the pre/post split; these are read off the indicator. The same convention scales without new arguments – when several units are treated at different times, dataprep() cohorts them by their individual adoption dates automatically. This is a deliberate contrast with the surrounding ecosystem. The widely used scpi package (Cattaneo *et al.* 2025) asks the analyst to name the treated unit and to delimit the post-treatment period explicitly; the forward-selection fsPDA implementation (Shi and Huang 2023) requires the data to be pre-pivoted into wide treated-versus-donor matrices before it will run. In mlsynth, coding the indicator column *is* the specification, and the same long data frame drives every one of the more than forty estimators unchanged.

3.2. One configuration-and-fit contract

Each estimator is driven by a dedicated configuration object built on Pydantic (Colvin and Pydantic Contributors 2024), a runtime type-validation library. Configurations forbid unknown fields, document every option, and fail early with a typed error when an input is malformed – a missing treatment column, a treatment date outside the sample, an empty donor pool. There are no free-form keyword arguments. The usage pattern is uniform: construct a configuration (here as a plain dictionary, which Pydantic coerces and validates), pass it to the estimator class, and call fit().

```
from mlsynth import VanillaSC

config = {
```

```
    "df": panel,            # a long/tidy pandas DataFrame
    "outcome": "cigsale",   # the outcome column
    "treat": "treat",       # 0/1 treatment indicator
    "unitid": "state",      # the unit column
    "time": "year",         # the time column
}
results = VanillaSC(config).fit()
```

Switching estimators means changing the class name and, at most, a handful of method-specific options; the data, the column names, and the call shape do not move. This is the concrete payoff of the unified view in Section 2.

3.3. One result contract

Every `fit()` returns a typed result object drawn from a single hierarchy. Observational estimators return an effect result; design methods return a design result whose report is itself an effect result. Both populate the same standardized sub-objects – estimated effects, fit diagnostics, the observed and counterfactual time series, donor weights, and inference – as typed fields rather than ad hoc dictionaries. Because the result shape is fixed, downstream code (plotting, tables, comparison across estimators) is written once and works for every method. The two illustrations below read effects, time series, and inference off this common object without any estimator-specific glue.

4. A single treated unit

We begin with the study that made the method famous. In 1988 California passed Proposition 99, a large tobacco-control program; Abadie *et al.* (2010) estimated its effect on per-capita cigarette sales by building a synthetic California from other states.

A modelling choice that the methods literature takes seriously concerns the donor pool. Abadie *et al.* (2010) deliberately discard control units whose own policies contaminate the counterfactual: four states that adopted large-scale tobacco programs of their own during the sample (Massachusetts, Arizona, Oregon, Florida), seven that raised cigarette taxes by fifty cents or more (Alaska, Hawaii, Maryland, Michigan, New Jersey, New York, Washington), and the District of Columbia. The remaining thirty-eight states are the canonical Abadie-Diamond-Hainmueller sample – the same panel the synthetic-difference-in-differences study of Arkhangelsky *et al.* (2021) later adopted. The robust, principal-component approach of Amjad *et al.* (2018) makes the opposite choice on purpose: its singular-value thresholding is designed to find a good donor subset on its own, so it keeps every available control and lets the de-noising do the selection. We load the canonical restricted sample for the first two estimators and the full fifty-state-plus-DC panel for the third.

```
# Canonical Abadie-Diamond-Hainmueller sample: California and the 38 control
# states that remain after dropping the contaminated controls.
abadie = pd.read_csv(find_data("smoking_data.csv"))
abadie["treat"] = abadie["Proposition 99"].astype(int)
```

```
# The full pool -- every state plus DC -- for the principal-component estimator.
full = pd.read_csv(find_data("prop99_with_dc.csv"))
full = full[full["year"] <= 2000].copy()
full["treat"] = ((full["state"] == "California") & (full["year"] >= 1989)).astype(int)

sc_base = dict(df=abadie, outcome="cigsale", treat="treat",
               unitid="state", time="year")
pcr_base = dict(df=full, outcome="cigsale", treat="treat",
                unitid="state", time="year")

n_donors_sc = abadie["state"].nunique() - 1
n_donors_pcr = full["state"].nunique() - 1
```

4.1. Three designs, one interface

We fit three members of the family. The first is the canonical simplex-weighted synthetic control, **VanillaSC**, on Abadie's 38-state pool, with the prediction intervals of Cattaneo, Feng, and Titiunik (2021). The second is synthetic difference-in-differences (Arkhangelsky *et al.* 2021), a popular design that adds unit and time weights to a two-way fixed-effects fit, exposed as **SDID**. The third is the principal-component (outcome-only) variant of **CLUSTERSC**, which de-noises the donor matrix by truncating its singular-value spectrum – the low-rank reading of Equation 1 in Section 2.1 – run on the full 50-donor pool.

The point of this illustration is how little separates the three. We describe each estimator only briefly here – the optimization problem and inference behind each are documented per estimator at https://mlsynth.readthedocs.io – because the illustration is about the interface they share, not the internals they do not. Setting `display_graphs=True` asks each estimator to draw its own diagnostic in the library's style. Figure 1 is the simplex control's plot; `inference="scpi"` shades the Cattaneo *et al.* (2021) prediction band over the post-treatment window automatically.

```
vsc = VanillaSC({**sc_base, "inference": "scpi", "alpha": 0.10,
                 "display_graphs": True}).fit()
att_vsc = float(vsc.effects.att)
```

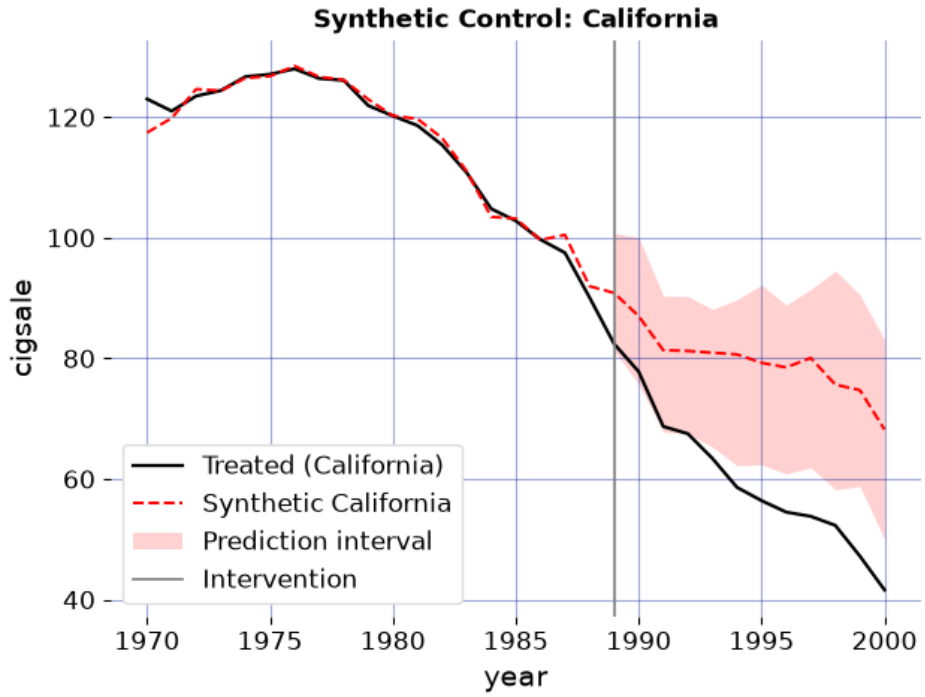

Figure 1: Simplex synthetic control on Abadie's canonical 38-state donor pool, drawn by VanillaSC itself. Observed California cigarette sales versus the synthetic control, with the Cattaneo-Feng-Titiunik 90% prediction band over the post-treatment window. The solid vertical line marks the 1988 passage of Proposition 99.

Swapping to synthetic difference-in-differences takes a single edit: the *same* `sc_base` dictionary, with the estimator class changed from **VanillaSC** to **SDID**. Nothing about the data, the column names, or the call shape moves. Here `B=2000` sets the number of placebo replications behind SDID's standard error. Two further options are worth naming. With a single treated unit **SDID** draws an observed-versus-counterfactual chart in the same shared style as the other two estimators (Figure 2); a staggered design with several adoption cohorts would instead return its pooled event-study plot. And `intercept_adjust=True` shifts the reported counterfactual by the SDID intercept – the constant level difference between California and its weighted donors – so that it is level-matched to the treated unit over the pre-period, as the original presentation of Arkhangelsky *et al.* (2021) shows it. By default `mlsynth` reports the raw weighted-donor series instead; the point estimate and inference are the same either way.

```
sdid = SDID({**sc_base, "display_graphs": True, "B": 2000,
             "intercept_adjust": True}).fit()
att_sdid = float(sdid.effects.att)
sdid_se = float(sdid.inference.standard_error)
sdid_ci_lo = float(sdid.inference.ci_lower)
sdid_ci_hi = float(sdid.inference.ci_upper)
```

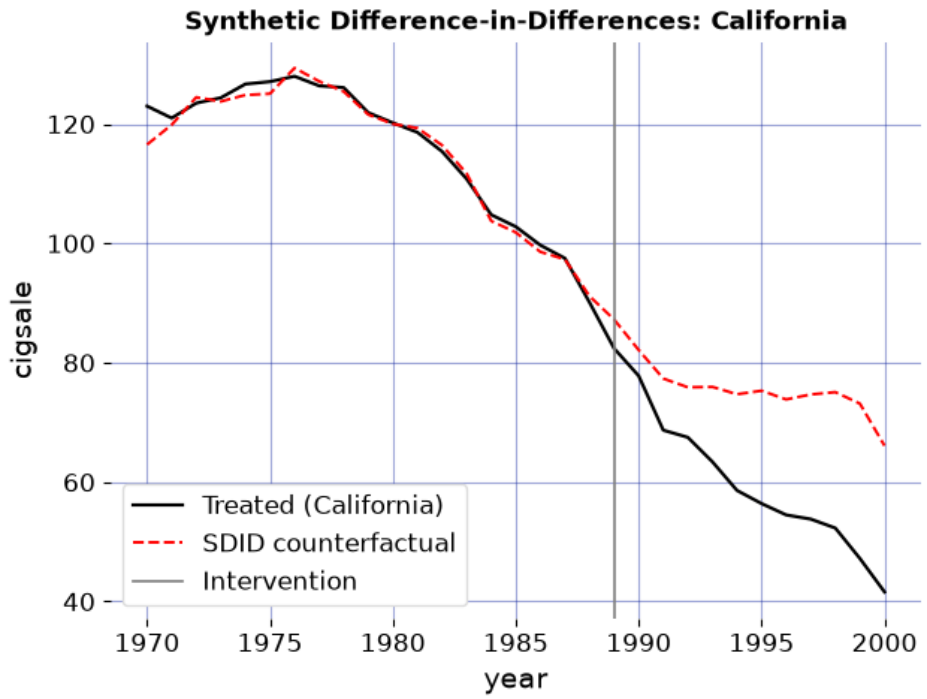

Figure 2: Synthetic difference-in-differences on the same 38-state pool, reached by swapping the estimator class. Observed California cigarette sales versus the intercept-adjusted SDID counterfactual; the solid vertical line marks the 1988 passage of Proposition 99.

The third design keeps every donor. Figure 3 is the principal-component estimator's plot, fit on the full pool and drawn in the same observed-versus- counterfactual style as the simplex control. It does not return a period-by-period band; following Amjad *et al.* (2018) it reports a confidence interval on the average effect, read off the standardized result below.

```
pcr = CLUSTERSC({**pcr_base, "method": "pcr", "display_graphs": True}).fit()
att_pcr = float(pcr.effects.att)
pcr_ci_lo = float(pcr.inference.ci_lower)
pcr_ci_hi = float(pcr.inference.ci_upper)
```

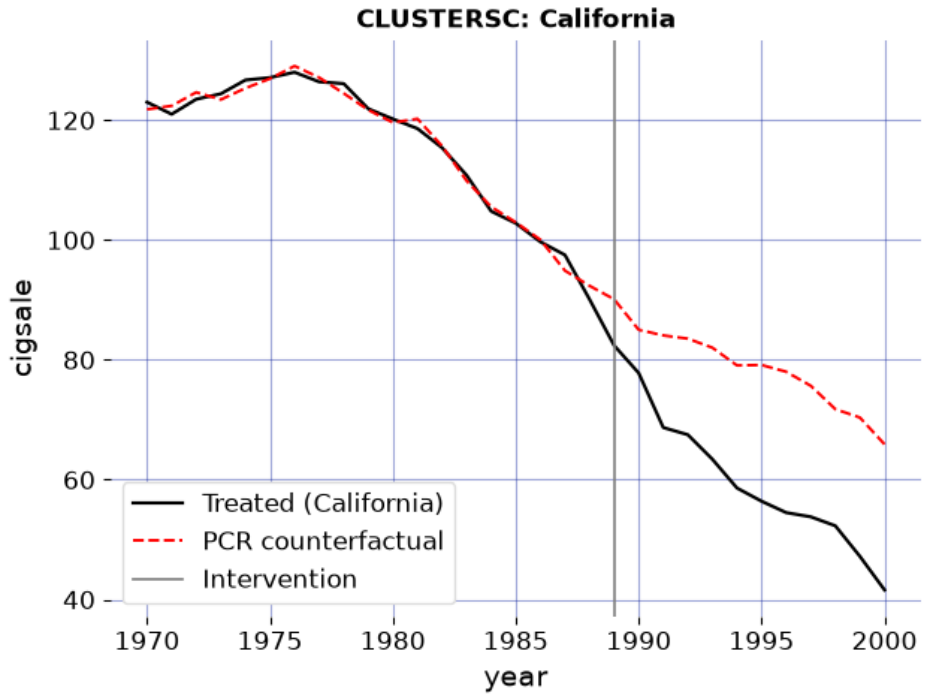

Figure 3: Principal-component (HSVT) synthetic control on the full donor pool, drawn by CLUSTERSC itself. Observed California cigarette sales versus the principal-component counterfactual; the solid vertical line marks the 1988 passage of Proposition 99.

The three designs agree on the sign and broad magnitude of the effect while differing in detail, exactly as their literatures would predict. The simplex synthetic control puts the average post-1988 effect at -19.51 packs per capita; synthetic difference-in-differences, which leans on a fixed-effects baseline, gives a smaller -15.61 (standard error 7.37; 95% placebo confidence interval -30.04 to -1.17) – reproducing the headline estimate of Arkhangelsky *et al.* (2021) to the first decimal; and the principal-component variant lands at -18.37 packs, with a confidence interval running from -20.22 to -16.53. All three say California smoked substantially less than its synthetic counterpart after Proposition 99. None of this required reformatting or reconciliation: the three results expose the same `effects.att` field, the figures came from the same `display_graphs` switch, and moving between a simplex control, a synthetic difference-in-differences design, and a robust principal-component estimator was a change of class name against an unchanged data frame. That is the whole argument of the paper in miniature – swapping designs is a one-line edit, not a one-package migration.

The donor weights are part of the standardized result as well, under `weights.donor_weights`, and they reward inspection because the two designs reach their answers through visibly different weightings. The simplex control concentrates all its mass on a handful of states (Table 2): synthetic California is a sparse convex combination, the hallmark of the unit-simplex constraint in Equation 3. Synthetic difference-in-differences, whose unit weights carry an ℓ_2 ridge penalty, instead spreads mass thinly across many donors (Table 3). Both mappings are read the same way, off the same field.

```
sc_weights = pd.Series(vsc.weights.donor_weights, name="weight")
```

```
sc_weights = sc_weights[sc_weights.abs() > 1e-4].sort_values(ascending=False)
sc_weights.round(3).rename_axis("donor state").to_frame()
```

donor state	weight
Utah	0.394
Montana	0.232
Nevada	0.205
Connecticut	0.109
New Hampshire	0.045
Colorado	0.015

Table 2: Donor weights of the simplex synthetic control, read from `vsc.weights.donor_weights`. Synthetic California is a sparse convex combination of a few states; donors with zero weight are omitted.

```
sdid_weights = pd.Series(sdid.weights.donor_weights, name="weight")
sdid_weights.sort_values(ascending=False).head(8).round(3).rename_axis("donor state").to_f
```

donor state	weight
Nevada	0.124
New Hampshire	0.105
Connecticut	0.078
Delaware	0.070
Colorado	0.057
Illinois	0.053
Nebraska	0.048
Montana	0.045

Table 3: The eight largest SDID unit weights, read from `sdid.weights.donor_weights`. The ridge penalty spreads weight across many donors rather than concentrating it.

5. Spillovers

The illustration so far treats the donor pool as clean: untreated units are assumed unaffected by the treatment, the stable-unit-treatment-value assumption (SUTVA) that synthetic control inherits from the broader causal-inference literature. That assumption is often indefensible. When California raised its cigarette tax, some sales simply crossed the border into Nevada, so a neighbouring state's outcomes were themselves moved by the treatment; using it as a donor contaminates the counterfactual. For most of the method's history the remedy was subtractive – drop the units you suspect, as Abadie *et al.* (2010) do – trading a SUTVA violation for a smaller, weaker comparison. A more recent literature instead keeps the units and models the spillover: Cao and Dowd (2023), Di Stefano and Mellace (2024), Melnychuk (2024), and

Grossi, Mariani, Mattei, Lattarulo, and Öner (2025) each estimate the direct effect and the leakage jointly.

These four estimators arrived as four separate codebases, in another language, with four data conventions. `mlsynth` exposes them through one dispatcher, **SPILLSYNTH**, selected by a `method` keyword; the treated unit, the donor pool, and the units suspected of absorbing spillover are declared once. The demonstration we want is therefore not of any single estimator but of the consolidation: with a common interface, comparing four spillover-aware methods across two unrelated case studies is a loop, not a porting project. We run all four on the California tobacco panel – with the dozen neighbours Abadie *et al.* (2010) discard reinstated as suspected spillover recipients – and on West Germany's 1990 reunification (Abadie, Diamond, and Hainmueller 2015), with Austria, France, and the Netherlands as the recipients, plotting each method's spillover-adjusted counterfactual against the observed treated unit.

```
adh_covariates = ["trade", "infrate", "industry", "schooling",
                  "invest60", "invest70", "invest80"]
cases = [
    dict(name="Proposition 99", file="prop99_with_dc.csv", span=(1970, 2000),
         outcome="cigsale", unit="state", time="year", treated="California",
         t0=1989, ylab="cigarette sales (packs p.c.)", covariates=None,
         affected=["Alaska", "Arizona", "District of Columbia", "Florida",
                   "Hawaii", "Massachusetts", "Maryland", "Michigan",
                   "New Jersey", "Nevada", "New York", "Oregon", "Washington"]),
    dict(name="Reunification", file="scpi_germany.csv", span=(1960, 2003),
         outcome="gdp", unit="country", time="year", treated="West Germany",
         t0=1990, ylab="GDP p.c. (000s USD)", covariates=adh_covariates,
         affected=["Austria", "France", "Netherlands"]),
]
methods = {"cd": ("C0", "-"), "iscm": ("C3", "--"),
           "iterative": ("C1", "-."), "grossi": ("C4", ":")}

fig, axes = plt.subplots(1, 2, figsize=(7, 3))
records = []
for ax, case in zip(axes, cases):
    df = pd.read_csv(find_data(case["file"]))
    df = df[df[case["time"]].between(*case["span"])].copy()
    df["treat"] = ((df[case["unit"]] == case["treated"])
                   & (df[case["time"]] >= case["t0"])).astype(int)
    cfg = dict(df=df, outcome=case["outcome"], treat="treat", unitid=case["unit"],
               time=case["time"], affected_units=case["affected"],
               display_graphs=False)
    years = sorted(df[case["time"]].unique())
    obs = (df[df[case["unit"]] == case["treated"]]
           .sort_values(case["time"])[case["outcome"]].to_numpy())
    ax.plot(years, obs, color="black", lw=1.6, label="observed")
    for m, (color, ls) in methods.items():
        kw = {**cfg, "method": m}
        if m == "iscm" and case["covariates"]:
```

```
            kw["covariates"] = case["covariates"]      # only iscm matches on covariates
        fit = SPILLSYNTH(kw).fit()                      # the one line that varies
        sub = getattr(fit, m)
        pre = (sub.a[0] + sub.B[0] @ fit.inputs.Y_pre if m == "cd"
               else sub.treated_synthetic_pre)
        ax.plot(years, list(pre) + list(sub.counterfactual_sp),
                color=color, ls=ls, lw=1.2, label=m)
        records.append({"case": case["name"], "method": m, "ATT": fit.att})
    ax.axvline(case["t0"], color="0.4", ls="-", lw=1.0)
    ax.set_title(case["name"], fontsize=8)
    ax.set_xlabel("year"); ax.set_ylabel(case["ylab"], fontsize=8)
axes[0].legend(fontsize=6, ncol=2)
fig.tight_layout()
plt.show()
```

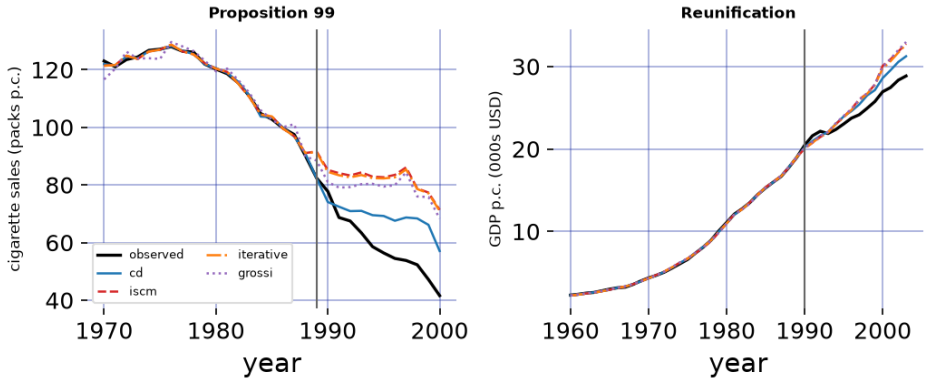

Figure 4: Four spillover-aware estimators on two case studies, reached by changing one `method` keyword in a loop. Each panel plots the observed treated unit (black) against each method's spillover-adjusted synthetic counterfactual. On Proposition 99 (left) the methods disagree substantially – spillover into the reinstated neighbours is large; on German reunification (right) they nearly coincide – the declared spillover is small. The spread within a panel is a cross-method sensitivity check that the shared interface makes free.

```
(pd.DataFrame(records)
   .pivot(index="method", columns="case", values="ATT")
   .round(2))
```

case method	Proposition 99	Reunification
cd	-9.44	-0.98
grossi	-19.01	-1.53
iscm	-22.25	-1.46
iterative	-21.70	-1.38

case	Proposition 99	Reunification
method		

Table 4: Average treatment effect on the treated for each spillover-aware estimator on each case study, read from the same `att` field. On Proposition 99 the estimates span roughly nine to twenty-two packs; on reunification they cluster within a few hundred dollars of one another.

The two panels make the same point from opposite directions. On Proposition 99 the four methods disagree by more than a factor of two, because the treatment genuinely leaks into the reinstated neighbours and each estimator apportions that leakage differently; on reunification, where only three modest neighbours are declared affected, the corrections are small and the four counterfactuals nearly coincide. The disagreement is information, not defect: its size tracks how much spillover there is to model, and reading it off costs one loop over a `method` keyword rather than four installations across two languages. As with the single-unit illustration, `mlsynth` takes no position on which estimator is correct – that judgement belongs to the analyst and the application. What the package supplies is the cheaper, prior thing: the ability to run all of them, on any of your panels, behind one interface.

6. Experimental design

The second illustration moves to the design question, where `mlsynth` reaches capability that the surrounding ecosystem has not packaged for practitioners. We reproduce the public walkthrough of Meta's `GeoLift` framework (Meta Open Source 2024), a widely used industry tool for geographic marketing experiments. The walkthrough ships a panel of 40 markets observed over 105 days and asks: if we run a geo-experiment in two cities, can we recover the lift, and is it significant? `GeoLift` answers this with an augmented synthetic control (Ben-Michael *et al.* 2021) and conformal inference (Chernozhukov, Wüthrich, and Zhu 2021).

We load the same public test panel, mark the final 15 days as the post-intervention window, and drive the **GEOLIFT** estimator exactly as a user would – forcing the walkthrough's two treated markets (Chicago and Portland) through the configuration.

```
geo = pd.read_csv(find_data("geolift_test_data.csv"))
dates = sorted(geo["date"].unique())
PRE = 90
geo["post"] = geo["date"].isin(set(dates[PRE:])).astype(int)


def geolift_fit(augment):
    cfg = {
        "df": geo, "outcome": "Y", "unitid": "location", "time": "date",
        "treatment_size": 2, "to_be_treated": ["chicago", "portland"],
        "durations": [len(dates) - PRE], "effect_sizes": [0.0, 0.10],
        "lookback_window": 1, "post_col": "post", "how": "mean",
        "augment": augment, "fixed_effects": True, "power_threshold": 0.8,
        "alpha": 0.1, "ns": 2000, "seed": 0, "conformal_type": "iid",
```

```
        "display_graphs": False,
    }
    return GEOLIFT(cfg).fit()
```

`GeoLift`'s walkthrough prints two summaries: a base, unaugmented model and a ridge-augmented "best" model. We run both through the one interface.

```
base_geo = geolift_fit(None).report
ridge_geo = geolift_fit("ridge").report

geo_base_att = float(base_geo.effects.att)
geo_ridge_att = float(ridge_geo.effects.att)
geo_p = float(base_geo.inference.p_value)
```

6.1. Reproducing the published numbers

The walkthrough reports an average treatment effect of 155.556 for the base model and 156.805 for the ridge-augmented model, with a conformal p-value rounding to 0.01. `mlsynth`, driven through its public interface, returns 155.556 and 156.805 for the two models and a p-value of 0.015. These match the published figures to the third decimal – the point estimate, the augmentation, and the inference all reproduce. The full benchmark, which additionally pins the percent lift, the L2 imbalance, the scaled imbalance, the percent improvement over the naive model, and all thirteen donor weights against the vignette's printed tables, ships with the library as a durable regression test.

	GeoLift (published)	mlsynth
Base ATT	155.556	155.556
Ridge-augmented ATT	156.805	156.805
Conformal p-value	0.010	0.015

Table 5: mlsynth reproduces the GeoLift walkthrough's published summaries through its public interface.

The donor weights are exposed exactly as in the estimation illustration, under `weights.donor_weights`. For the ridge-augmented model they reproduce the markets and weights printed in the vignette's own weight table (Table 6) – Cincinnati, Miami, Baton Rouge, and the rest of the synthetic test region.

```
geo_weights = pd.Series(ridge_geo.weights.donor_weights, name="weight")
geo_weights = geo_weights[geo_weights > 1e-4].sort_values(ascending=False).head(13)
geo_weights.round(4).rename_axis("donor market").to_frame()
```

donor market	weight
cincinnati	0.2273
miami	0.2029

donor market	weight
baton rouge	0.1337
minneapolis	0.0901
dallas	0.0741
nashville	0.0687
honolulu	0.0674
austin	0.0467
san diego	0.0452
reno	0.0308
san antonio	0.0056
houston	0.0048
new york	0.0048

Table 6: Donor-market weights of the ridge-augmented GeoLift model, read from `ridge_geo.weights.donor_weights`, matching the walkthrough's printed weight table. Markets with negligible weight are omitted.

That `mlsynth` reproduces `GeoLift` exactly is the starting point, not the finish. The point-fit estimator and the conformal inference are held fixed to match `GeoLift`'s methodology faithfully – they are not knobs to be swapped for arbitrary alternatives. What `mlsynth` adds is a far richer language for constraining *which markets the experiment may treat*. Through the same configuration dictionary, the analyst can forbid or force particular markets, declare interference clusters and a spillover-adjacency matrix so that mutually contaminating markets are never split across the test and control groups, impose stratum coverage quotas (at least and at most so many treated markets per region, tier, or segment), restrict the treated set to a size band, and cap the design by cost-per-incremental-conversion and budget. These are formulations the upstream tool does not expose, reached without leaving the interface.

6.2. The broader design family

The `GeoLift` reproduction is one entry point into a family of experimental-design estimators that `mlsynth` packages together. Where `GeoLift` fixes the treated units and asks whether an effect can be recovered, the design methods of Doudchenko and Imbens (2016) and Doudchenko *et al.* (2021) invert the problem: given a pool of candidate units and a target effect size, they *choose* which units to treat so that a credible synthetic control will exist afterward, and report the statistical power of the resulting design. `mlsynth` exposes these as peers of the estimation methods – the same configuration-and-fit shape, the same result contract – so that selecting an experiment and, later, analyzing it are two calls against one library rather than two separate tools. To our knowledge, no other package places this design family alongside the full estimation catalog behind a common interface.

7. Computational details and reproducibility

All numbers and figures in this paper are computed when the document is rendered, directly from the installed library, rather than transcribed from a prior run. This is a deliberate guard against the drift that accumulates when a paper's reported figures and a library's behavior diverge over successive versions: if an estimator's output changed, this document would fail to render or would render different numbers, not silently disagree with itself.

The results above were produced with `mlsynth` 1.0.0 on Python 3.13.14, with NumPy 2.5.0 (Harris *et al.* 2020) and pandas 3.0.3. The convex weight problems are solved with `CVXPY` (Diamond and Boyd 2016). The package and its dependencies are open source; `mlsynth` is available from the Python Package Index via `pip install mlsynth`, with source and issue tracker at https://github.com/jgreathouse9/mlsynth and documentation at https://mlsynth.readthedocs.io.

7.1. A controlled validation suite

Reproducibility in `mlsynth` is engineered, not asserted. Alongside the unit tests, the library ships a separate benchmark suite that institutionalizes the cross-checks that are otherwise performed once by hand and discarded. Each estimator is validated along one of three paths: reproducing a source paper's empirical result on the authors' own data, reproducing a paper's Monte Carlo table, or – most demandingly – matching an authoritative reference implementation cell by cell.

The cross-validation path is where the engineering shows. Rather than copy a reference tool's published numbers into a test (numbers that silently rot as the reference evolves), the suite installs the reference implementation itself and runs it at validation time, comparing `mlsynth`'s output to the freshly computed reference. Because most of these references are R packages – `augsynth`, `GeoLift`, `microsynth`, `Synth`, `pensynth`, among others – and R packages under active development drift, every reference is pinned to an exact commit. The install scripts compile each package, and its entire dependency chain, from a frozen Git SHA, so the same reference code runs every single time. The `GeoLift` cross-check, for instance, pins `augsynth` and its solver stack (`osqp`, `S7`, `LiblineaR`) to specific commits; the script that does so notes, pointedly, that an unpinned tip is precisely the version drift that made `GeoLift`'s own walkthrough ATT of 155.556 go stale. The Python case shells out to the pinned reference, compares the two outputs to a tight numerical tolerance – often four significant figures – and reports a pass or fail, skipping cleanly when the R toolchain is unavailable rather than masquerading as a pass. The `GeoLift` reproduction in Section 6 is exactly such a case: the published figures it matches are regenerated, not remembered, from `augsynth` frozen at a known commit. Reproducibility is thereby turned into a controlled, re-runnable process in which both sides of every comparison are fixed and inspectable.

8. Summary

Synthetic control has grown from a single method into a sprawling family, and the software has fragmented along with it. We have argued that the fragmentation is largely incidental: most of these estimators are variations on one factor-model projection, differing in how they constrain or denoise the same underlying fit. `mlsynth` takes that observation seriously, placing

more than forty estimators behind one data contract, one validated configuration-and-fit interface, and one standardized result. The three illustrations show what the consolidation buys. Estimation becomes a genuine comparison – a simplex synthetic control, a synthetic difference-in-differences design, and a robust principal-component estimator on the California data, each run as its authors prescribed yet reached by a one-line change of class against an unchanged data frame. Relaxing the no-interference assumption becomes a loop: four spillover-aware estimators, otherwise four separate codebases in another language, run across two case studies behind one keyword. And design becomes available at all: the library reproduces Meta's `GeoLift` walkthrough to within numerical tolerance and then opens it onto a broader experimental-design family that the surrounding ecosystem has never packaged for applied users. The value of having all of this in one place is not convenience but capability: a single, validated interface is what makes forty methods comparable, and what lets one library answer both the question of what an intervention did and the question of how to design the experiment that will measure it.

References

Abadie A (2021). "Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects." *Journal of Economic Literature*, **59**(2), 391–425. doi:10.1257/jel.20191450.

Abadie A, Diamond A, Hainmueller J (2010). "Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program." *Journal of the American Statistical Association*, **105**(490), 493–505. doi:10.1198/jasa.2009.ap08746.

Abadie A, Diamond A, Hainmueller J (2011). "**Synth**: An R Package for Synthetic Control Methods in Comparative Case Studies." *Journal of Statistical Software*, **42**(13), 1–17. doi:10.18637/jss.v042.i13.

Abadie A, Diamond A, Hainmueller J (2015). "Comparative Politics and the Synthetic Control Method." *American Journal of Political Science*, **59**(2), 495–510. doi:10.1111/ajps.12116.

Abadie A, Gardeazabal J (2003). "The Economic Costs of Conflict: A Case Study of the Basque Country." *American Economic Review*, **93**(1), 113–132. doi:10.1257/000282803321455188.

Amjad M, Shah D, Shen D (2018). "Robust Synthetic Control." *Journal of Machine Learning Research*, **19**(22), 1–51.

Arkhangelsky D, Athey S, Hirshberg DA, Imbens GW, Wager S (2021). "Synthetic Difference-in-Differences." *American Economic Review*, **111**(12), 4088–4118. doi:10.1257/aer.20190159.

Athey S, Bayati M, Doudchenko N, Imbens G, Khosravi K (2021). "Matrix Completion Methods for Causal Panel Data Models." *Journal of the American Statistical Association*, **116**(536), 1716–1730. doi:10.1080/01621459.2021.1891924.

Bai J (2009). "Panel Data Models with Interactive Fixed Effects." *Econometrica*, **77**(4), 1229–1279. doi:10.3982/ECTA6135.

Ben-Michael E, Feller A, Rothstein J (2021). "The Augmented Synthetic Control Method." *Journal of the American Statistical Association*, **116**(536), 1789–1803. doi:10.1080/01621459.2021.1929245.

Cao J, Dowd C (2023). "Estimation and Inference for Synthetic Control Methods with Spillover Effects." *arXiv preprint arXiv:1902.07343*.

Cattaneo MD, Feng Y, Palomba F, Titiunik R (2025). "**scpi**: Uncertainty Quantification for Synthetic Control Methods." *Journal of Statistical Software*, **113**(2), 1–38. doi:10.18637/jss.v113.i02.

Cattaneo MD, Feng Y, Titiunik R (2021). "Prediction Intervals for Synthetic Control Methods." *Journal of the American Statistical Association*, **116**(536), 1865–1880. doi:10.1080/01621459.2021.1979561.

Chernozhukov V, Wüthrich K, Zhu Y (2021). "An Exact and Robust Conformal Inference Method for Counterfactual and Synthetic Controls." *Journal of the American Statistical Association*, **116**(536), 1849–1864. doi:10.1080/01621459.2021.1920957.

Clarke D, Pailañir D, Athey S, Imbens G (2023). "Synthetic Difference in Differences Estimation." *arXiv preprint arXiv:2301.11859*. doi:10.48550/arXiv.2301.11859.

Colvin S, Pydantic Contributors (2024). ***pydantic**: Data Validation Using Python Type Hints*.

Di Stefano R, Mellace G (2024). "The inclusive Synthetic Control Method." 2403.17624, URL https://arxiv.org/abs/2403.17624.

Diamond S, Boyd S (2016). "**CVXPY**: A Python-Embedded Modeling Language for Convex Optimization." *Journal of Machine Learning Research*, **17**(83), 1–5.

Doudchenko N, Imbens GW (2016). "Balancing, Regression, Difference-in-Differences and Synthetic Control Methods: A Synthesis." *NBER Working Paper*, (22791). doi:10.3386/w22791.

Doudchenko N, Khosravi K, Pouget-Abadie J, Lahaie S, Lubin M, Mirrokni V, Spiess J, Imbens G (2021). "Synthetic Design: An Optimization Approach to Experimental Design with Synthetic Controls." *Advances in Neural Information Processing Systems*, **34**.

Grossi G, Mariani M, Mattei A, Lattarulo P, Öner Ö (2025). "Direct and spillover effects of a new tramway line on the commercial vitality of peripheral streets: a synthetic-control approach." *Journal of the Royal Statistical Society Series A: Statistics in Society*, **188**(1), 223–240. doi:10.1093/jrsssa/qnae032.

Harris CR, Millman KJ, van der Walt SJ, *et al.* (2020). "Array Programming with NumPy." *Nature*, **585**, 357–362. doi:10.1038/s41586-020-2649-2.

Hsiao C, Ching HS, Wan SK (2012). "A Panel Data Approach for Program Evaluation: Measuring the Benefits of Political and Economic Integration of Hong Kong with Mainland China." *Journal of Applied Econometrics*, **27**(5), 705–740. doi:10.1002/jae.1230.

Melnychuk A (2024). "Synthetic Controls with spillover effects: A comparative study." 2405.01645, URL https://arxiv.org/abs/2405.01645.

Meta Open Source (2024). "GeoLift: An End-to-End Geo-Experimentation Methodology." https://github.com/facebookincubator/GeoLift.

Robbins MW, Davenport S (2021). "**microsynth**: Synthetic Control Methods for Disaggregated and Micro-Level Data in R." *Journal of Statistical Software*, **97**(2), 1–31. doi:10.18637/jss.v097.i02.

Shi Z, Huang J (2023). "Forward-Selected Panel Data Approach for Program Evaluation." *Journal of Econometrics*, **234**(2), 512–535. doi:10.1016/j.jeconom.2021.04.009.

Xu Y (2017). "Generalized Synthetic Control Method: Causal Inference with Interactive Fixed Effects Models." *Political Analysis*, **25**(1), 57–76. doi:10.1017/pan.2016.2.

Yan G, Chen Q (2022). "**rcm**: A Command for the Regression Control Method." *The Stata Journal*, **22**(4), 842–883. doi:10.1177/1536867X221140960.

Yan G, Chen Q (2023). "**synth2**: Synthetic Control Method with Placebo Tests, Robustness Test, and Visualization." *The Stata Journal*, **23**(3), 597–624. doi:10.1177/1536867X231195278.

Affiliation:

Jared Greathouse
Department of Economics, Andrew Young School of Policy Studies
Georgia State University
Atlanta, GA, United States
E-mail: jgreathouse3@student.gsu.edu
URL: https://jgreathouse9.github.io/